



Performance and Capacity Test Plan — TASCO Growth Platform

TASCO Insurance × VETC Growth Platform — Quality Engineering

iorta TechNXT

Version 1.0 · 07 October 2026

Contents

1. Objectives
2. Workload model
3. Scenarios
4. SLO targets
5. Tooling
6. Test environment (PERF)
7. Monitoring during tests
8. Pass / fail criteria
9. Capacity model
10. Schedule

Item	Value
Owner	iorta TechNXT Performance Engineer
Reviewers	TASCO IT (infrastructure), VETC IT (app traffic forecasts), SRE
Related	Test strategy §2.6 · Monitoring and alerting · Known issues KI-04, KI-05, KI-12
Status	Baseline v1.0. Every workload figure marked (A) is an assumption and must be confirmed by VETC/TASCO with real data at G1 and re-baselined after the first 2 pilot weeks

1. Objectives

1. Show that the platform meets the latency and throughput SLOs (§4) at the forecast peak, with headroom $\geq 2\times$.
2. Show that it can hold and process the **full VETC base of ~6 million vehicle profiles**: initial load, incremental daily rebuilds, full lead recompute, and daily journey runs within their batch windows.
3. Find the breaking point and confirm graceful degradation (429/503, breakers open, no data corruption).
4. Find leaks over long runs (soak).
5. Produce a **capacity model** (§9) that converts business volumes into pods, database size and connection counts, so ops can scale before each wave.

2. Workload model

2.1 Population

Quantity	Value	Basis
Vehicle profiles (golden records)	6,000,000	VETC ETC base (programme brief)
VETC app users among them	70–80 % → 4.2–4.8 M	Programme brief
Source records per vehicle	~1.5 → **~9 M** source rows	(A) several sources per vehicle (VETC account, partner records, TASCO core)
Vehicles with usable expiry at start	~10 % (≈ 600 k)	Brief ("1 in 10 has a valid stamp"); grows as customers and the bot confirm
Expiries per day (uniform)	6 M / 365 $\approx$ 16,400	Annual TNDS term
Peak renewal day	3× average $\approx$ 50,000 expiries	(A) clustering at month end, after Tết and on registration anniversaries

2.2 Daily batch volumes (steady state at full scale)

Flow	Daily volume	Derivation
Touchpoints due	~115,000 (peak ~300,000)	~6–7 renewal and conquest steps per expiring vehicle (<code>journeys.json</code>) + lapsed and new-vehicle journeys
Messages attempted (push / ZNS / SMS)	~100,000 (peak ~280,000)	~1 send per digital touchpoint, plus fallbacks

Flow	Daily volume	Derivation
Voice bot calls	~6,500 (peak ~20,000)	Hot/warm share (≈ 40 %) of expiring vehicles at step -14 days; call consent required
Telesales handoffs	~1,500	≈ 22 % of calls end hot_handoff (simulated persona mix in simulatedCaller.js) + journey escalations
Incremental ingest	60,000–180,000 records	(A) 1–3 % of the base changes per day (new tags, inspections, partner records)
Lead recompute	Incremental (event-driven) + weekly full 6 M	profiles.rebuilt, lead.recompute_requested; full npm run job -- recompute
Reconciliation	All orders	npm run job -- reconcile nightly

2.3 Online traffic

Actor	Peak-hour assumption	Requests per session	Peak RPS (sustained)	Burst RPS
Customers (VETC app / Zalo mini app)	Peak day: 75,000 sessions, 15 % in the peak hour → 11,250 sessions/h (A)	~6 (session , home , quotes , orders , home , consent)	~19	~ 100 for 10 min after a push blast (10 % of 100 k recipients open within 10 min)
Telesales and staff consoles	250 concurrent staff (200 agents) (A)	1 request / 10 s per user	~25	~50 at shift start
Partner API	30 partners in wave W2 (A)	—	quotes 20 TPS, orders 5 TPS, other 5 TPS (A)	50 TPS
Public certificate verification (QR)	Traffic police, inspection centres, customers (A)	1	5	20
Ecosystem events (POST /api/ecosystem/events)	Tag activations, inspection bookings, top-ups (A)	1	10	50
Total design point			~100 RPS sustained	~300 RPS design, 600 RPS stress target (2×)

Partner TPS figures are assumptions, not commitments. They size the test until partners provide forecasts. Contractual per-partner rate limits are enforced at the API gateway.

2.4 Transaction mix for the mixed-load scenario (PERF-S1)

Group	Endpoints	Share of requests
Customer reads	GET /api/customer/home , GET /api/customer/quotes	25 %
Customer quote	POST /api/customer/quotes (TNDS, TNDS + PA_SEAT)	12 %

Group	Endpoints	Share of requests
Customer order	POST /api/customer/orders (unique Idempotency-Key)	4 %
Customer session	POST /api/customer/session (signed link)	5 %
Staff reads	GET /api/leads , GET /api/customers/:id , GET /api/handoffs , GET /api/touchpoints	20 %
Staff writes	PATCH /api/handoffs/:id , POST /api/quotes , POST /api/quotes/:id/send	6 %
Partner	POST /api/partner/v1/quotes , POST /api/partner/v1/orders , GET /api/partner/v1/policies	15 %
Public	GET /api/public/certificates/:certNo	5 %
Events	POST /api/ecosystem/events	5 %
Dashboards	GET /api/dashboard/overview	1 % (rate-capped: it is a heavy aggregate)
Login	POST /api/auth/login + /mfa	2 %

3. Scenarios

ID	Scenario	Load profile	Duration	Purpose	Catalogue
PE RF-S1	Peak-hour mixed load	Ramp 0 → 300 RPS over 10 min, hold 30 min	45 min	SLO verification at the design point	TC-136
PE RF-S2	Push-blast spike	20 → 150 RPS customer traffic in 30 s, hold 10 min	15 min	Burst after a journey run sends ~100 k messages	—
PE RF-S3	Renewal-day purchase path	30 orders/s on /api/customer/orders , with wallet and TASCO core latency injected (p50 300 ms, p95 900 ms)	30 min	Idempotency and orders under load; breaker headroom	TC-073
PE RF-S4	Partner API burst	50 TPS partner mix from 30 keys	20 min	Key lookup cost (hash query per request); per-IP rate limits behind a gateway (TRUST_PROXY)	—
PE RF-S5	Inline-drain interference (KI-04)	PERF-S1 + a background event backlog of 50 k pending events	30 min	Measures how events.drain() inside requests inflates order and quote latency	—
PE RF-S6	Staff shift start	200 logins in 5 min (script N=16384) + console reads	10 min	CPU cost of password hashing; login rate limiting per IP when agents share a NAT egress IP (RATE_LIMIT_LOGIN_MAX =10/min)	TC-121
PE RF-S7	Volume: initial load of 6 M profiles	~9 M synthetic source records via the ingest service in batches of 2,000 (KI-16)	Until done	Load window and DB growth; counts reconcile	TC-137

ID	Scenario	Load profile	Duration	Purpose	Catalogue
PE RF- S8	Incremental daily rebuild	180 k changed records on top of 6 M	Until done	Must finish in ≤ 60 min (incremental by plate, chunks of 500)	—
PE RF- S9	Full lead recompute + daily journey run at 6 M	<code>npm run job -- recompute</code> , then <code>npm run job -- journeys</code> with 115 k due touchpoints	Until done	Recompute ≤ 4 h; journeys done within the contact window (08:00–20:00 ICT). See KI-05 and KI-12	—
PE RF- S10	Stress to failure	Step +100 RPS every 5 min from 300 until SLO breach or errors $> 5\%$	~60 min	Breaking point; degradation mode; recovery time	TC-139
PE RF- S11	Soak	60 % of the design point (≈ 180 RPS) + background relay + hourly journey runs	12 h (24 h before W1)	Memory and connection leaks; <code>process_resident_memory_bytes</code> slope	TC-138
PE RF- S12	Dependency degradation	PERF-S1 with the wallet at 50 % errors, then down	30 min	Breaker opens (<code>integration_short_circuit_total</code>), the rest of the platform stays within SLO	TC-079
PE RF- S13	DB failover under load	PERF-S1 + managed failover	30 min	Readiness flips, recovery ≤ 2 min, no 500 storm	TC-141

4. SLO targets

Latency is server-side, measured by `http_request_duration_seconds` (route label = route template) and confirmed client-side by the load generator. External integration time is included where the endpoint calls one; the **sandbox adds none**, so PERF tests inject realistic latency into the sandbox ports.

Route group	Routes	p95	p99	Error budget (5xx)
Health and public	<code>/health/*</code> , <code>GET /api/meta</code> , <code>GET /api/public/certificates/:certNo</code>	≤ 150 ms	≤ 300 ms	$< 0.1\%$
Customer reads	<code>GET /api/customer/home</code> , <code>GET /api/customer/claims</code>	≤ 300 ms	≤ 600 ms	$< 0.1\%$
Quotes	<code>POST /api/customer/quotes</code> , <code>POST /api/quotes</code> , <code>POST /api/partner/v1/quotes</code> (known plate)	≤ 500 ms	≤ 1 s	$< 0.1\%$
Partner quote, unknown plate (onboarding + rebuild)	<code>POST /api/partner/v1/quotes</code>	≤ 1.2 s	≤ 2.5 s	$< 0.5\%$
Orders (pay + issue)	<code>POST /api/customer/orders</code> (wallet + core), <code>POST /api/partner/v1/orders</code> (core only, partner-collected)	≤ 2.5 s with real integrations (≤ 800 ms platform time excluding wallet and core)	≤ 5 s	$< 0.5\%$ excluding upstream 503

Route group	Routes	p95	p99	Error budget (5xx)
Staff reads	GET /api/leads , GET /api/customers/:id , GET /api/handoffs , GET /api/touchpoints , GET /api/policies	≤ 500 ms	≤ 1 s	< 0.5 %
Staff writes	PATCH /api/handoffs/:id , POST /api/voice/sessions/:id/turns , PUT /api/customer/consent	≤ 500 ms	≤ 1 s	< 0.5 %
Auth	POST /api/auth/login , /mfa	≤ 600 ms	≤ 1.2 s	< 0.1 %
Dashboards	GET /api/dashboard/*	≤ 3 s at 6 M (KI-12)	≤ 6 s	< 1 %
Batch	Incremental rebuild 180 k records	≤ 60 min	—	0 failed batches
Batch	Full recompute 6 M	≤ 4 h	—	—
Batch	Daily journey run	All due touchpoints executed by 12:00 ICT	—	Skipped only for policy reasons

Throughput targets: **300 RPS** mixed at SLO with CPU ≤ 70 % on the planned replica count; **600 RPS** without data corruption (SLO breach allowed).

5. Tooling

Tool	Use
<code>npm run test:perf → test/perf/load.js</code>	Dependency-free Node HTTP load generator in the repo: weighted scenario mix, concurrency/RPS control, reports p50/p95/p99 latency, RPS and error counts per route . Target URL, duration, concurrency and mix are configured as documented in the script header. Used in CI as a perf smoke and in PERF for S1–S6, S10–S13
k6 (optional)	Distributed load from several injectors when one Node process cannot generate the target RPS, or when the client side must model think time and VU sessions. Scripts mirror the <code>load.js</code> mix
Synthetic generator	<code>syntheticVetcSource.generate({count, seed, today})</code> for 6 M records, generated in province-sized chunks so the generator's memory stays bounded
Batch jobs	<code>npm run job -- migrate seed recompute journeys reconcile relay</code> timed with <code>/usr/bin/time -v</code> ; job history from <code>GET /api/ops/jobs</code>
Fault injection	Sandbox ports wrapped with latency and error rate (the wallet and notification sandboxes accept <code>failRate</code>); <code>tc netem</code> or Toxiproxy for DB latency
Database	<code>pg_stat_statements</code> , <code>EXPLAIN (ANALYZE, BUFFERS)</code> on top queries, <code>pg_stat_activity</code> for pool saturation

6. Test environment (PERF)

Component	PERF sizing	Rationale
App pods	3 → 10 (HPA on CPU 60 %), each 1 vCPU / 1 GiB request, 2 vCPU / 1.5 GiB limit; <code>DB_POOL_MAX=10</code>	Mirror PROD; Node runs one event loop per pod

Component	PERF sizing	Rationale
Relay	Built-in 1 s relay in every pod (<code>server.js</code>) + <code>relay</code> CronJob	As PROD
PostgreSQL 16	8 vCPU / 32 GiB / 500 GB SSD (3k+ IOPS); <code>max_connections</code> 200 or PgBouncer in transaction mode	Holds 6 M profiles + indexes with headroom (§9.3)
Load injectors	2 × 4 vCPU in the same region, outside the cluster	Avoid measuring the injector
Integrations	Sandbox adapters with injected latency (wallet p50 300 ms / p95 900 ms; core issue p50 500 ms / p95 1.5 s; ZNS 200 ms)	Real partners are not load-tested without written agreement
Configuration	<code>NODE_ENV=production</code> , <code>DEMO_MODE=false</code> , <code>LOG_LEVEL=info</code> , <code>TRUST_PROXY=true</code> , raised <code>RATE_LIMIT_MAX</code> for injector IPs (or inject through the gateway with X-Forwarded-For)	The per-IP limiter would otherwise throttle the injectors

7. Monitoring during tests

Signal	Source	Watch for
Request rate, errors and latency per route	<code>http_requests_total</code> , <code>http_request_duration_seconds_bucket</code>	p95 per route vs §4; 5xx and 429
Integration health	<code>integration_calls_total{result}</code> , <code>integration_short_circuit_total</code>	Retries inflating calls; breaker opening
Event flow	<code>events_published_total</code> , <code>events_processed_total{status}</code> , <code>GET /api/ops/status</code> → <code>eventBacklog</code>	Backlog growth, <code>dead_letter</code>
Business throughput	<code>quotes_total</code> , <code>orders_completed_total</code> , <code>messages_total{status}</code> , <code>voice_calls_total</code>	Matches the injected mix
Process	<code>process_resident_memory_bytes</code> , <code>process_uptime_seconds</code> (restarts), container CPU throttling	Leak slope; OOM kills
Database	CPU, IOPS, <code>pg_stat_activity</code> active/idle-in-tx, lock waits (advisory lock 724001 for audit appends), slowest statements	Pool saturation; audit-append serialisation becoming the bottleneck
Logs	JSON <code>request</code> lines (<code>route</code> , <code>status</code> , <code>ms</code> , <code>requestId</code>)	Tail latency outliers by <code>requestId</code>

8. Pass / fail criteria

Criterion	Pass
SLOs	All route groups meet the p95 and p99 in §4 at 300 RPS (PERF-S1, S3, S4)
Errors	5xx < 0.1 % (excluding injected upstream faults); no <code>INTERNAL_ERROR</code> in logs
Resources at the design point	App CPU ≤ 70 % , DB CPU ≤ 60 % , no pool waits > 100 ms p95
Spike (S2)	p95 back within SLO ≤ 2 min after the spike starts; HPA scales out
Volume (S7–S9)	6 M profiles loaded; counts reconcile; incremental rebuild ≤ 60 min; journey run completes in the window (fails today by design: see KI-05)

Criterion	Pass
Stress (S10)	Degradation via 429/503 only; zero corrupted orders (reconciliation clean afterwards); recovery ≤ 5 min
Soak (S11)	RSS growth < 5 %/h after a 1 h warm-up; no restarts; flat error rate
Resilience (S12, S13)	Wallet outage isolates sales only; failover recovery ≤ 2 min
Regression	Release-candidate p95 within 120 % of the previous baseline (CI perf smoke)

Every run produces a report: build/commit, environment, data volume, mix, `load.js` summary (p50/p95/p99, RPS, errors per route), Grafana snapshot links, DB top statements, findings and tuning actions.

9. Capacity model

9.1 Online tier

```

pods_needed = ceil( peak_RPS × weighted_CPU_ms_per_request / (1000 × target_util) )

```

with `target_util` = 0.6. Calibrate `weighted_CPU_ms_per_request` from PERF-S1. Planning value (A): ~4 ms of event-loop time per request on average. Logins are excluded: one script hash costs ~50–80 ms CPU. Orders are excluded too: they are I/O-bound (wallet + core).

Wave	Peak RPS (design)	Planning pods (1 vCPU)	Min / max replicas (HPA)	DB connections (pods × <code>DB_POOL_MAX</code>)
Pilot (HN + HCM, 100–200 k vehicles)	30	2 → 3 (HA floor)	3 / 4	30–40
W1 nationwide car TND5 (6 M base)	300	~2 by CPU → 4 for headroom and AZ spread	4 / 10	40–100 → PgBouncer
W2 + partner API	350	5	5 / 12	PgBouncer required

9.2 Batch tier

The current implementation processes batch work **sequentially in one process**. That is the dominant capacity constraint at 6 M, not the online tier.

Job	Work per item (current code)	Items	Estimate @ 1 ms per DB round trip	Window	Status
Journey run	~7 queries + one sequential gateway call (ZNS/SMS 200–500 ms real) per touchpoint; limit 5,000 per run	115 k/day (peak 300 k)	115 k × ~0.3 s ≈ 9.6 h sequential; peak > 24 h	08:00–20:00 ICT	Does not fit at peak. Needs pagination beyond 5,000, deferral of window-blocked items, and concurrent sends (KI-05)
Full lead recompute	3–8 queries per profile + OFFSET paging	6 M	6 M × ~5 ms ≈ 8 h , worse with OFFSET (KI-12)	Weekly, overnight	At risk. Keyset pagination + parallel workers by region
Incremental rebuild	~4 queries per plate + DQ upserts; chunks of 500	60–180 k	≈ 4–15 min	Hourly or daily	OK

Job	Work per item (current code)	Items	Estimate @ 1 ms per DB round trip	Window	Status
Initial load	As above	6 M plates / 9 M records	≈ 8–12 h	Weekend, split by province	OK with planning
Reconciliation	1 + N policy lookups per order; OFFSET paging	Orders (grows ~10 k+/day)	Minutes in year 1	Nightly	OK, revisit at 5 M orders
Audit verify	Reads the whole audit_log into memory	~70 M rows/yr (A)	Memory-bound; will fail eventually (KI-12)	On demand	Needs checkpointing before W1

9.3 Storage (year 1, 6 M base)

Table	Rows	Avg row (data + index) (A)	Size
profiles	6 M	4 KB (encrypted PII, lineage)	~24 GB
leads	6 M	2.5 KB (reasons, benefits)	~15 GB
source_records	9 M (365-day retention)	1.2 KB	~11 GB
touchpoints	~42 M/yr	0.7 KB	~29 GB
messages	~36 M/yr	0.9 KB (encrypted to)	~32 GB
domain_events	~20 M/yr (no retention yet: KI-13)	0.6 KB	~12 GB
audit_log	~70 M/yr (A)	0.5 KB	~35 GB
voice_sessions	~2.4 M/yr (180-day retention)	3 KB (encrypted transcript)	~4 GB
Orders, quotes, policies, handoffs, claims	~10 M/yr	1 KB	~10 GB
Total			~170 GB + WAL/PITR (≈ 2× daily churn × 7 days) → provision 500 GB, alert at 70 %

9.4 Scaling triggers

Trigger	Action
p95 of any group > 80 % of SLO for 3 days	Add 1 replica to the HPA minimum; profile the hot route
DB CPU > 60 % at peak or connections > 70 % of max	Introduce or resize PgBouncer; scale the DB vertically; add a read replica for dashboards
Journey run not finished by 12:00 ICT	Split runs per journey/region (multiple journeys jobs with distinct dates) until KI-05 is fixed
eventBacklog.pending > 10 k sustained	Increase relay frequency or replicas; investigate slow handlers

Trigger	Action
Disk > 70 %	Expand storage; implement retention for events and audit archive (KI-13)

10. Schedule

When	Runs
Sprint 4 (Dec 2026)	Baseline PERF-S1, S6 on synthetic 500 k; establish CI perf smoke
SIT/UAT window (late Dec 2026 – Jan 2027)	Full S1–S13 at 6 M before G3; report to SteerCo
Pilot weeks 1–2	Re-baseline the workload model with real traffic (replace every (A))
Before W1 (Mar 2027)	Repeat S7–S11 at full scale with fixes for KI-04/05/12; 24 h soak
Each release	CI perf smoke; full S1 if the release touches sales, journeys or persistence